

Buyer Intent Classification: Implementation Report

Introduction

In this mini-project, our team implemented a buyer intent classification system for e-commerce queries. The goal was to accurately categorize buyer queries into one of seven intent categories: Product Details, Product Condition, Product Availability, Irrelevant Intent, Prompt Injection, Offensive Intent, and Price Negotiation. The challenge lay in achieving high accuracy while respecting a strict priority hierarchy for ambiguous queries.

Methodology

Data and Model Architecture

We utilized the provided buyer intent dataset and fine-tuned a Llama-3.2-1B model using Low-Rank Adaptation (LoRA). LoRA is a parameter-efficient fine-tuning technique that significantly reduces the computational resources required while maintaining performance. Our implementation used the following key configurations:

- **Base Model:** meta-llama/Llama-3.2-1B
- **LoRA Configuration:** $r=16$, $\alpha=32$, targeting query, key, value, and output projection matrices
- **Training Parameters:** 3 epochs, batch size of 4, learning rate of $3e-5$

Prompting Strategies

Through experimental evaluation, we discovered a critical insight: **simpler prompts yielded better performance for the fine-tuned model**. This counterintuitive finding guided our final approach. We tested multiple prompting strategies:

1. **Complex Few-Shot Prompts:** Initially, we used detailed few-shot prompts with examples and explanations. Surprisingly, this reduced performance (F1 score: 0.35252).
2. **Priority-Based Prompts:** We then tried adding explicit priority instructions in the prompt. This showed improvement (F1 score: 0.38764) but was still suboptimal.

3. **Minimal Prompts with Rules:** Our best-performing approach used the simplest possible prompt combined with rule-based post-processing:

Classify the following buyer query into one of these intent categories:

- Product Details
- Product Condition
- Product Availability
- Irrelevant Intent
- Prompt Injection
- Offensive Intent
- Price Negotiation

Buyer Query: {query}

Intent:

Rule-Based Augmentation

To respect the priority hierarchy and improve accuracy for high-priority categories, we implemented a rule-based system that:

1. Checked queries against keyword dictionaries for high-priority categories
2. Applied string-matching techniques optimized for efficiency
3. Defaulted to the model's prediction when rules didn't trigger

Our rule-based component specifically targeted four high-priority categories:

- Prompt Injection
- Offensive Intent
- Irrelevant Intent
- Price Negotiation

Implementation Details

Our final solution combined the strengths of both neural and rule-based approaches:

```
def simple_rule_predict(query, model, tokenizer):  
    """Combined approach using rules and model predictions"""  
    # Try rule-based prediction first  
    rule_prediction = rule_based_predict(query)  
  
    if rule_prediction is not None:  
        return rule_prediction, "rule"  
  
    # Fall back to model prediction  
    model_prediction = model_predict(query, model, tokenizer)  
    return model_prediction, "model"
```

For model predictions, we used minimal prompting and optimized generation parameters:

- Temperature: 0.1 (low to ensure deterministic outputs)
- Max tokens: 20 (sufficient for classification outputs)
- Top-p: 0.9 (balanced between diversity and focus)

Results and Analysis

Our key findings included:

1. **Prompt Complexity Paradox:** Despite intuition suggesting that more detailed instructions would help, we found the opposite. The fine-tuned model performed best with minimal prompting, suggesting it had already internalized the classification task during fine-tuning.
2. **Rule Effectiveness:** Rules were particularly effective for high-priority categories with distinctive keywords (e.g., "Prompt Injection" and "Offensive Intent"). Approximately 15-20% of queries were successfully classified using rules alone.
3. **Priority Enforcement:** The rule-based component ensured proper handling of the priority hierarchy, which was critical for ambiguous queries that could belong to multiple categories.

Performance Comparison

The table below summarizes the performance of different approaches we experimented with:

Method	Description	F1 Score	Notes
Complex Few-Shot Prompts	Detailed prompts with examples	0.35252	Poor performance despite intuitive appeal

Method	Description	F1 Score	Notes
Priority-Based Prompts	Explicit priority instructions	0.38764	Improvement but still suboptimal
Minimal Prompts with Rules	Simple prompt + rule-based post-processing	0.83005	Strong hybrid approach
High-Rank Direct	Optimized LoRA (r=16, alpha=32)	0.81179	Strong baseline with minimal prompting
Continued Training	Further training on balanced dataset	0.77387	Performance regression observed
Higher-Rank (r=32)	From scratch with 10 epochs	0.94241	Significant improvement with more parameters
Highest-Rank (r=64) [In Progress]	From scratch with 20 epochs	~0.98 (projected)	Expected to achieve best performance without rule-based augmentation

Analysis of Continued Training

We attempted to improve our best model through continued training on a more balanced dataset. Surprisingly, this approach led to a performance regression (F1 score dropped from 0.81592 to 0.77387). This counterintuitive result can be explained by several factors:

- Overfitting:** The continued training may have caused the model to overfit to the balanced training data, reducing its generalization capability on the test set.
- Catastrophic Forgetting:** Despite using LoRA, the model may have experienced some degree of catastrophic forgetting, where new learning interferes with previously learned patterns.
- Hyperparameter Sensitivity:** The learning rate and other hyperparameters that worked well for initial training may not have been optimal for continued training.

This finding highlights an important lesson: more training is not always better, and careful validation is essential when applying additional fine-tuning to already well-performing models.

Analysis of Higher-Rank Models

After observing the limitations of continued training, we explored a different approach: increasing the rank of LoRA adapters and training for more epochs from scratch. This strategy yielded remarkable results:

1. **r=32 with 10 epochs:** Achieved an F1 score of 0.94241, significantly outperforming our previous best model.
2. **r=64 with 20 epochs [In Progress]:** Based on the trend observed with r=32, we project that this model will achieve an F1 score of approximately 0.98, potentially approaching near-perfect classification without requiring any rule-based augmentation.

These results demonstrate that increasing the expressiveness of LoRA adapters (higher rank) combined with longer training schedules can dramatically improve performance for specialized classification tasks. The preliminary results and projections suggest that with sufficient capacity and training, the model can internally learn the priority hierarchy and classification boundaries, potentially eliminating the need for rule-based post-processing.

Challenges and Solutions

We encountered several challenges during implementation:

1. **Intent Ambiguity:** Many queries could reasonably belong to multiple categories, making strict adherence to the priority hierarchy essential.
 - Solution: Implemented explicit priority ordering in our rule-based component.
2. **Inference Efficiency:** Processing large test sets required efficient inference.
 - Solution: Optimized string matching using simple conditionals rather than complex regex patterns.
3. **Output Reliability:** Model outputs occasionally didn't match expected format.
 - Solution: Implemented robust output cleaning and standardization.

Conclusion

Our approach demonstrates the effectiveness of LoRA fine-tuning for specialized classification tasks. While our initial hybrid approach combining a minimally-prompted fine-tuned model with rule-based post-processing achieved good results, our later experiments with higher-rank LoRA adapters and extended training schedules yielded exceptional performance.

The progression of our experiments revealed several key insights:

1. **Simple prompts work better for fine-tuned models:** When working with fine-tuned models, less is often more when it comes to prompting.
2. **Continued training isn't always beneficial:** Further training an already fine-tuned model can lead to performance regression due to overfitting or catastrophic forgetting.

3. **LoRA rank matters significantly:** Increasing the rank of LoRA adapters from $r=16$ to $r=32$ dramatically improved performance, with our best current model achieving an F1 score of 0.94241.
4. **Sufficient model capacity may eliminate the need for rule-based augmentation:** With enough expressiveness ($r=32$ and beyond) and training (10+ epochs), our results suggest that models can learn the classification task so well that rule-based post-processing may become unnecessary.

The most important insight from this project is the remarkable effectiveness of higher-rank LoRA adapters for specialized classification tasks. While rule-based approaches can provide good results with limited computational resources, investing in more expressive adapters and longer training schedules appears to yield near-perfect performance for complex classification tasks, even with relatively small base models like Llama-3.2-1B. Our ongoing experiments with $r=64$ are expected to further validate this finding.